Piece of History

Step-by-Step Replit Agent Build Prompts

Use these in order, in a single Replit Agent chat/session. Let each step finish and get verified before pasting the next one. Each prompt assumes the previous steps are already built.

STEP 1 — Kickoff & Architecture Plan

Start every build with a plan before any code gets written, so the agent commits to the right architecture up front.

Copy — paste into Replit Agent:

```
You are the lead product engineer for a Replit project called "Piece of History."
Core tagline: "Every day disappears. Some days become history."

Core mechanic: one "day" is active at a time. Each participant submits a short
story and an optional website URL, then makes real, non-refundable payments to
compete to be #1. A participant's rank is the SUM of every payment they have
made that day (a running total), not just their latest single payment. At
exactly 23:59:59 (server time, configurable timezone) the day freezes and
whoever has the highest running total wins. The win, and every payment
collected that day, goes to the site owner. No platform fee, no refunds for
non-winners, no proxy bidding.

Visual direction: "Snowy Blue" — white and pale icy-blue backgrounds, navy
text/buttons, subtle snowflakes and mountain illustrations, playful
micro-copy. Clean and fast, never visually heavy.

Do NOT build: social features (followers/likes/comments/chat), subscriptions,
points, badges, NFTs/crypto, AI-generated stories, a marketplace, or proxy
bidding. Keep this a tiny, memorable side project with one core mechanic.

Recommended stack: Next.js (App Router) + TypeScript + Tailwind CSS, Replit's
built-in PostgreSQL database with Prisma ORM, NextAuth.js for auth, Razorpay
for payments behind a small provider-abstraction layer.

Before writing any code: produce (1) a short architecture overview, (2) a full
database schema (Users, DailyPiece, Story, Bid-as-ledger, computed
ParticipantStanding, optional AuditEvent), and (3) a state-transition diagram
for a day going active → receiving payments → closing → archived. Wait for my
confirmation before implementing.
```

STEP 2 — Database Schema & Prisma Setup

Turns the agreed schema into real Prisma models and migrations, with the ledger + constraints that make the cumulative bidding logic safe.

Copy — paste into Replit Agent:

```
Set up Prisma with PostgreSQL (Replit's built-in database) for Piece of
History. Implement these models: User, DailyPiece, Story, Bid, and a way to
efficiently compute each user's running total per day (either a SQL view or a
maintained ParticipantStanding table kept in sync inside the same transaction
as each Bid insert).

Bid rows are an immutable payment ledger: id, user_id, daily_piece_id, amount,
created_at, payment_reference, payment_status. Never update or delete a
captured Bid row.

Add database-level constraints: a DailyPiece can only have one winner, only
one DailyPiece can be "active" at a time, and amounts must be positive. Add
indexes on (user_id, daily_piece_id) for the ledger since this is queried
constantly to compute standings.

Write and run the initial migration. Then write a short script/seed that
creates one sample active day so I can see it in the database.
```

STEP 3 — Authentication

Basic sign-up/sign-in so every participant and their payments can be tied to an identity.

Copy — paste into Replit Agent:

```
Add authentication using NextAuth.js: email/password sign-up and sign-in, plus
Google and GitHub OAuth as alternatives. On first sign-in, create a User row
with a display name the user can edit later. Keep display names public but
never expose email addresses anywhere in the public UI. Build simple Sign In /
Sign Up pages using the Snowy Blue direction (icy-blue background, navy
primary button, a small snowman illustration) matching this layout: email
field, password field, "Remember me," "Continue with Google / GitHub"
buttons, and a link to switch between sign in and sign up.
```

STEP 4 — Snowy Blue Design System & App Shell

Locks in the visual language once, in shared components, before building individual pages.

Copy — paste into Replit Agent:

```
Build the shared design system and app shell for Piece of History in the
"Snowy Blue" direction: mostly white / very pale icy-blue backgrounds, deep
navy for text and primary buttons, soft gradients, subtle snowflake and
snow-mountain motifs, and a playful but clean tone (no heavy illustrations
that slow the page down). Set this up as Tailwind theme tokens (colors,
radius, shadows) plus a few reusable components: NavBar (logo, Home / Explore
/ Competition / How It Works links, Sign In button), PrimaryButton,
StoryCard, CountdownTimer, and Footer (with a small “Developed by …” line
that intentionally reveals nothing about the real identity, plus live stats:
“Launched X days ago · Users: X · Pieces of History: X” pulled from the real
database — never hard-code these numbers).

Then build the Landing/Home page: headline “Some days are just too good to
keep to yourself…”, a one-line pitch, a “Start Your Story” CTA, a 4-step
strip (Create your story → Choose a date → Let people bid → Win & make it
history), and today's #1 leader shown live.
```

STEP 5 — Participate Flow (Story + Website + First Payment)

The entry point where a user joins today's competition.

Copy — paste into Replit Agent:

```
Build the "Create Your Story" / Participate flow: a form with title/display
name, short story text, and an optional website URL (validate it's a proper
URL and block dangerous protocols like javascript:). On submit, create a
Story row linked to the active DailyPiece and to the signed-in user, then take
the user straight into the payment step to make their first real payment
(minimum amount configurable, e.g. $1). Clearly state on this screen, in plain
language, that every payment is real and non-refundable whether or not they
end up #1 — this must not be buried in fine print.
```

STEP 6 — Cumulative Bidding Engine + Razorpay Payments

The core mechanic: every payment is real, charged immediately, and adds to a running total that determines rank.

Copy — paste into Replit Agent:

```
Implement the cumulative real-money bidding engine for Piece of History:

1. Integrate Razorpay checkout for a payment of any amount ≥ the configured
minimum. Never mark a payment as successful from the client — only a verified
server-side Razorpay webhook may mark a Bid as captured.

2. When a payment is verified as captured, in a single database transaction:
insert the immutable Bid row, recompute that user's running total for the day
(SUM of all their captured Bids today), and recompute the day's current
leader (highest running total; on an exact tie, whoever reached that total
earliest keeps or gains #1). Use row-level locking on the DailyPiece so two
simultaneous payments can never both believe they are #1.

3. Example to satisfy: A pays $5 (A leads at $5). B pays $10 (B leads at $10).
A pays $6 more (A's total is now $11, so A leads again). Write an automated
test that reproduces exactly this sequence and asserts the leader at each
step.

4. Also write tests for: a payment below the minimum increment is rejected, a
duplicate/replayed webhook cannot double-count a payment, and two concurrent
payments that would tie are resolved deterministically.
```

STEP 7 — Live Competition Page

Where visitors watch the competition and click through to participants' sites.

Copy — paste into Replit Agent:

```
Build the "Today's Competition" page: a prominent #1 leader card (display
name, story snippet, running total, a big “Visit Website” button if they gave
one), a live countdown to the day's closing_timestamp (server time, updates
every second client-side but re-synced from the server periodically), and
below it a ranked list of every other participant with the same info and
their own “Visit Website” button (open in a new tab with rel="noopener
noreferrer"). Every participant, not just the leader, must have their website
link visible and clickable — this page is meant to send real traffic to
participants. Include a “Place a Payment” button that starts the flow from
Step 6. Poll the server every few seconds to refresh standings and rank
without a full page reload.
```

STEP 8 — Day-Close Job (23:59:59 Freeze & Winner Selection)

The scheduled job that ends the day and locks in the winner, safely.

Copy — paste into Replit Agent:

```
Implement the day-close operation as an idempotent database transaction:
given an active DailyPiece whose closing_timestamp has passed, lock the row,
determine the winner (highest running total, tie-break by earliest timestamp
reaching that total), set status to "closed," record winner_user_id and
winning_standing, set closed_at, and create tomorrow's new active DailyPiece.
Make sure running this twice (e.g. due to a retry) never changes the winner
or creates a duplicate active day.

Wire this up as a Replit Scheduled Deployment (cron) that calls this
operation every minute and only acts once the closing_timestamp has actually
passed. Write a test that calls the close operation twice in a row and
asserts the result is identical both times.
```

STEP 9 — Public Archive Pages

Permanent pages for every day that has already closed.

Copy — paste into Replit Agent:

```
Build the public Archive: a simple browsable list/calendar of all past
DailyPiece records, and an individual Archived Piece page per date showing
date, winner display name, winning story, winning standing, participant
count, and the winner's website link (still clickable, still rel="noopener
noreferrer"). These pages must never be editable through any normal user
action once the day is closed — enforce this at the API layer, not just in
the UI.
```

STEP 10 — My History Page

A private page where a user can see their own record across every day they joined.

Copy — paste into Replit Agent:

```
Build a private “My History” page (signed-in users only) listing every
DailyPiece the current user participated in: their story for that day, every
individual payment they made (amount + timestamp), their final running total,
their final rank, and whether they won or lost. Order most recent first and
paginate if the list is long. Do not add any social elements to this page —
it is private to the user, not a public profile.
```

STEP 11 — Minimal Admin View

Just enough visibility for the owner to operate the site — nothing more.

Copy — paste into Replit Agent:

```
Build a minimal, owner-only admin view (protect it with a role check, not just
a hidden URL) showing: a table of successful payments (user, day, amount,
payment reference, status), a table of closed days with their winners and
winning standings, and basic counts (total users, total pieces of history).
Do not build a general dashboard, analytics suite, or content moderation
system beyond a simple “hide this story” action for abuse cases.
```

STEP 12 — Security, Rate Limiting & Automated Tests

Hardening pass before this touches real money.

Copy — paste into Replit Agent:

```
Do a security and correctness pass on Piece of History:
- Add rate limiting on the payment-webhook endpoint and on story submission.
- Validate and sanitize all user input, especially the website URL field
(block javascript:, data:, and other unsafe protocols).
- Confirm HTTPS-only cookies, CSRF protection on state-changing routes, and
that no payment secrets or database credentials are exposed to the client.
- Confirm the countdown and closing decision use server time only, never a
value sent from the browser.
- Write/confirm automated tests for the critical rules: a higher cumulative
total overtakes the leader; a payment below the minimum is rejected; a
replayed webhook cannot double-count; concurrent payments cannot corrupt
the leaderboard; closing a day twice is safe; a closed day cannot be
modified; My History stays accurate after all of the above.
```

STEP 13 — Deployment on Replit

Ship it.

Copy — paste into Replit Agent:

```
Prepare Piece of History for production on Replit:
- Move all secrets (database URL, NextAuth secret, Razorpay keys and webhook
secret) into Replit Secrets, not hard-coded values.
- Set up a Replit Autoscale Deployment for the Next.js app and a Replit
Scheduled Deployment for the day-close job from Step 8.
- Add a health-check endpoint and structured server logs.
- Write a README covering: local setup, environment variables, running
Prisma migrations, configuring the daily closing timezone, setting up the
Razorpay webhook URL, switching from Razorpay test/sandbox keys to live
keys, and rollback steps.
- Confirm the app runs correctly end-to-end with Razorpay test/sandbox
payments before I switch on live payments.
```